



Projet Fondement de l'informatique

Clemente Iwan

Picard Marius

Moulin Yann

10 décembre 2025

1. Introduction
 - 1.1. Contexte du projet
 - 1.2. Objectifs du pilote automatique F1
 - 1.3. Contraintes et enjeux principaux
2. Méthodologie et outils
 - 2.1. Environnement de développement
 - 2.2. Organisation et répartition du travail
 - 2.3. Utilisation des IA
3. Architecture générale du pilote
 - 3.1. Structure modulaire
 - 3.2. Cycle de décision
 - 3.3. Communication avec le Gestionnaire de Course
4. Approche algorithmique globale
 - 4.1. Stratégie de course
 - 4.2. Heuristique et validation
 - 4.3. A* dans l'espace 4D
 - 4.4. Gestion des boosts et des surfaces spéciales
 - 4.5. Synthèse
5. Points forts et spécificités du pilote
 - 5.1. Un pilote qui finit ses courses
 - 5.2. Un A* adapté au temps réel
 - 5.3. Une architecture modulaire
6. Difficultés rencontrées et solutions apportées
 - 6.1. Gestion du carburant
 - 6.2. Coût du A* en temps
 - 6.3. Gestion des boosts
7. Résultats et validation
 - 7.1. Méthodologie de test
 - 7.2. Stabilité du comportement
 - 7.3. Qualité des trajectoire
 - 7.4. Limites
8. Conclusion

1.Introduction

1.1 Contexte du projet

Le projet Pilote Automatique de cette fin d'année propose un défi assez particulier : concevoir, en C, un pilote capable de participer à une course de Formule 1 virtuelle. Le programme doit interagir en temps réel avec un Gestionnaire de Course (GDC), il reçoit les positions des trois voitures et doit renvoyer une accélération (a_x , a_y) dans un délai strict d'une seconde.

1.2 Objectifs du pilote automatique F1

L'objectif est simple, il faut atteindre la ligne d'arrivée le plus vite possible sans :

- sortir de la piste (cases $.$, $\sim$, $=$, $\#$) ;
- tomber à carburant nul ;
- dépasser la limite de vitesse sur sable ($|v| \leq 1$).

Le pilote doit donc être capable de planifier des mouvements cohérents, et s'adapter aux imprévus. Le tout en respectant la contrainte stricte d'une seconde de calcul par tour.

1.3 Contraintes et enjeux principaux

Le projet impose des contraintes qui structurent fortement l'approche algorithmique : langage C99 avec flags stricts, physique du jeu variable selon le terrain, et consommation de carburant dépendant à la fois de la vitesse et de l'accélération. Ensemble, elles transforment ce qui semble être un pathfinding en un problème avec plusieurs critères d'optimisation.

Ce rapport présente notre démarche, nos choix de conception et les résultats obtenus.

2. Méthodologie et outils

2.1 Environnement de développement

Le développement du pilote s'est fait dans un environnement volontairement simple, afin de rester fidèle aux contraintes du projet. Le code est écrit en C99, compilé avec les options strictes imposées (-ansi -pedantic -Wall -Wextra). Nous avons utilisé un Makefile minimaliste pour automatiser la compilation et éviter les erreurs liées aux commandes manuelles. L'exécution du pilote s'est faite en interaction directe avec le Gestionnaire de Course (GDC), ce qui nous a permis de tester rapidement les comportements du programme et d'ajuster les paramètres au fur et à mesure.

2.2 Organisation générale du travail

La première répartition a été d'implémenter chacun une partie: Iwan s'est occupé de faire le A*, qui se base sur le Dijkstra inversé de Marius qui calcule l'heuristique. Yann s'est occupé des implémentations diverses comme les fonctions de communication avec le GDC ou la file de priorité avec des nœuds. Notre manière de travailler est restée assez spontanée, une fois les fonctions basiques implémentées, nous avons travaillé en alternance sur la quasi-totalité des fonctions et des améliorations de code.

Le projet avançant par itérations successives, nous avons régulièrement alterné entre phases d'implémentation, tests sur le GDC, et ajustements. Cette approche incrémentale nous a permis d'améliorer progressivement la stabilité du pilote et de valider nos choix au fur et à mesure, sans attendre la fin du développement.

2.3 Utilisation des IA

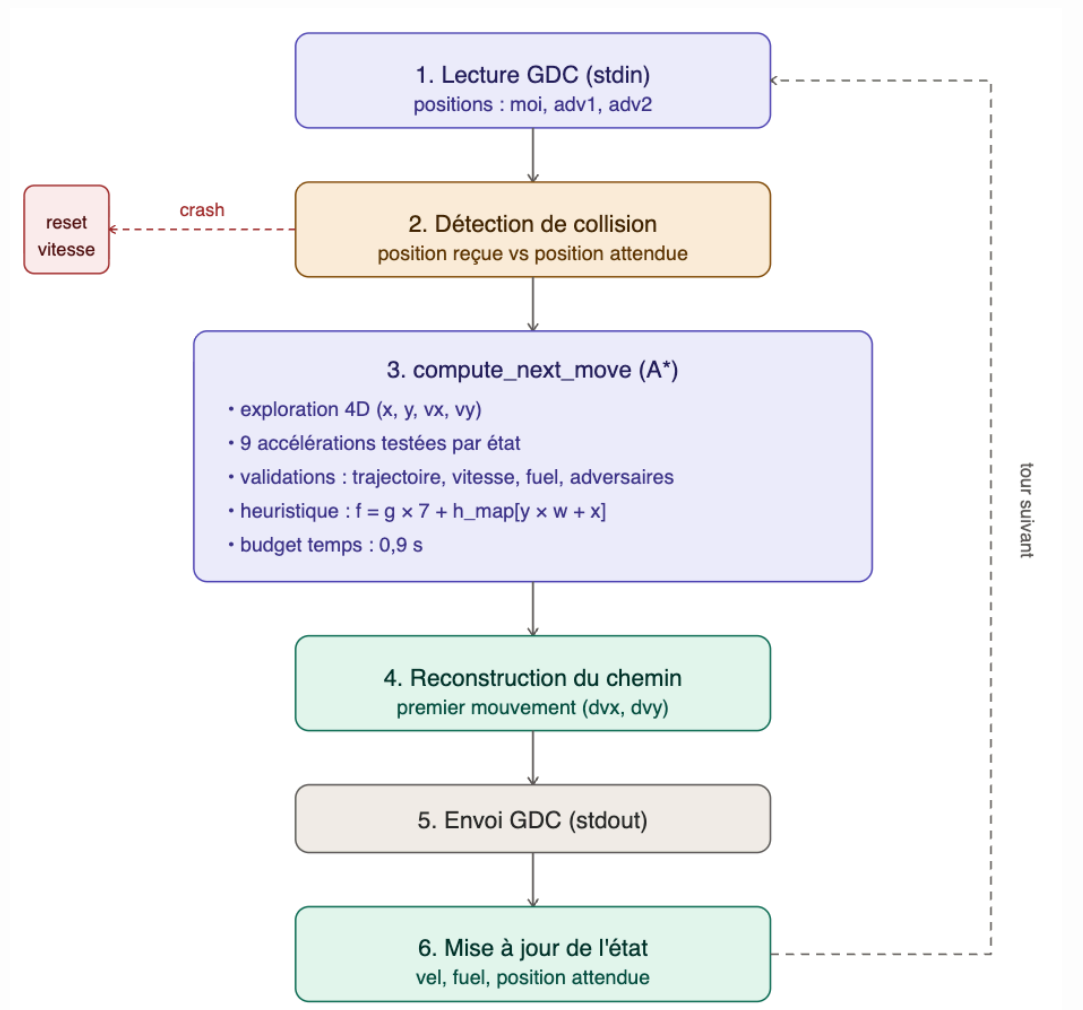
Comme l'utilisation des IA génératives étaient autorisées pour le développement de ce projet, nous avons décidé de tirer partie de cet outil puissant, sans se laisser submerger. En effet, l'implémentation de notre code a été en partie générée par IA. Cela nous a permis de gagner du temps sur les parties purement techniques, pour nous focaliser sur les questions de stratégie d'optimisation. Évidemment, nous mesurons le danger de cette pratique dans ce projet, et dans notre futur métier, et nous nous sommes forcés à vérifier le code produit par IA.

3. Architecture générale du pilote

3.1 Structure modulaire du programme

```
|— Makefile
|— README.md
|— bin
|— gdc
|— include
|— src
  |— communication.c
  |— dijkstra.c
  |— heap.c
  |— main.c
  |— map.c
  |— path_search.c
  |— physic.c
  |— prio_queue.c
```

3.2 Cycle de décision à chaque tour



3.3 Communication avec le Gestionnaire de Course

Le pilote communique exclusivement via `stdin` et `stdout`.

Au lancement, le GDC envoie :

- la largeur, la hauteur et le carburant initial
- la carte complète, ligne par ligne

À chaque tour, le GDC envoie 6 entiers :

- `myX myY` — position du joueur
- `adv1X adv1Y` — position du premier adversaire
- `adv2X adv2Y` — position du second adversaire

Le pilote lit ces valeurs, calcule son accélération, puis renvoie :

- `dvx dvy` — la variation de vitesse choisie

4. Approche algorithmique globale

4.1 Stratégie de Course

Notre pilote repose sur une approche algorithmique centrée sur la recherche de trajectoire optimale, tout en respectant les contraintes physiques et énergétiques du jeu. L'idée est que chaque accélération doit être cohérente avec la vitesse actuelle, la surface du terrain et le carburant disponible.

Pour ça, on a choisi de combiner deux algorithmes bien connus — A^* et Dijkstra — mais adaptés à notre contexte. A^* gère la recherche dynamique à chaque tour, tandis que Dijkstra sert à pré-calculer une heuristique fiable depuis la ligne d'arrivée.

En addition à cette recherche de trajectoire. Nous avons choisi une stratégie de prudence sur l'utilisation du carburant.

4.2 Heuristique et validation

La partie Dijkstra est exécutée une seule fois au démarrage : elle calcule, pour chaque case de la carte, la distance minimale vers la ligne d'arrivée. Les cases de sable sont pénalisées (coût +7), ce qui pousse naturellement le pilote à éviter ces zones.

Cette heuristique est ensuite utilisée par A* pour estimer la distance restante à chaque étape. Pour normaliser les valeurs, on a choisi de multiplier le coût g, qui représente le coût en nombre de tours, par 7 (qui correspond à la distance parcourue à la vitesse maximale théorique), afin de pouvoir l'additionner avec h, qui correspond à la distance restante à l'arrivée.

Chaque mouvement proposé par A* est ensuite validé par la fonction `is_move_valid`, qui vérifie toutes les positions intermédiaires entre le point de départ et le point d'arrivée du mouvement. Elle garantit que le pilote ne "saute" jamais une case hors piste ou ne traverse pas d'adversaires, même sur des trajectoires rapides.

4.3 A* dans l'espace 4D

On implémente un algorithme A*, mais avec plusieurs subtilités afin de répondre au mieux aux problèmes du sujet.

Premièrement, on se place dans un espace en 4 dimensions. En effet, on traite la position x, y, puis la vitesse vx et vy. C'est indispensable car notre voiture possède de l'inertie, donc arriver sur la même case avec une vitesse ou bien sans changer considérablement le futur de la course.

De plus, au lieu de simplement marquer si un nœud a déjà été visité ou non, on a choisi de stocker le niveau d'essence restant une fois cet état atteint. Cela a pour conséquence que si notre algorithme retombe sur un nœud qu'il a déjà visité à la même vitesse, il regardera si le nouveau chemin y arrive avec au plus autant d'essence restante. Si c'est le cas, le nouveau chemin sera ignoré. Cela permet de conserver les chemins économes afin de s'assurer que l'algorithme aura assez d'essence pour finir la course dans tous les cas.

Pour marquer si les nœuds ont été visités ou non, on utilise un tableau 1D au lieu de 4D, afin d'éviter de devoir faire des malloc à chaque tour. Pour pouvoir passer d'une grille 4D à un tableau en 1D, on utilise une formule "d'aplatissement" qui calcule un index :

$$index = (((y * largeur_grille + x) * V_SIZE + (vxi + 5)) * V_SIZE) + (vyi + 5).$$

Comme v_{xi} et v_{yi} sont compris entre -5 et 5, il faut les ramener dans un intervalle positif afin de s'en servir comme index du tableau, d'où le +5. V_size est défini comme le nombre de valeurs possibles pour chaque composante de vitesse, soit 11. Ensuite, dans chaque case on stocke un id de recherche et le fuel.

Au lieu de remettre à 0 toutes les structures du tableau à chaque tour, on choisit d'attribuer à chaque tour un id de recherche, qui s'incrémente de 1 par tour. Ainsi, si l'algorithme tombe sur une case qui contient un id antérieur à celui du tour actuel, il est tout simplement ignoré. Cela permet d'éviter de devoir remettre des cases à zéro à chaque tour, et de le faire uniquement si le tableau est rempli.

Comme l'algorithme explore des milliers de possibilités, il va forcément passer plusieurs fois sur la même case avec le même état, donc la même vitesse. Dans ce cas là, au lieu d'ignorer le nouveau chemin parce qu'il a déjà été visité comme ça serait le cas dans un A* classique, on compare d'abord l'essence restante en suivant le nouveau chemin avec celle restante du chemin déjà enregistré, qui est donc stockée dans le tableau *visited*, grâce à une structure. Si le nouveau chemin arrive à la même case dans le même état en ayant plus d'essence, alors il écrase l'ancien chemin dans le tableau.

4.4 Gestion des boosts et des surfaces spéciales

Notre pilote n'utilise pas de boosts. Nous expliquons pourquoi dans la section problème. Quant aux surfaces spéciales, elles sont traitées par le dijkstra qui distribue des heuristiques. Le sable reçoit un coût supplémentaire. Les murs reçoivent un coût infini.

4.5 Synthèse

En résumé, notre approche algorithmique repose sur une recherche A* dans un espace d'état 4D, prenant en compte le carburant utilisé et guidée par une heuristique Dijkstra inversée. Chaque décision est validée par la physique du jeu et filtrée par la consommation de carburant.

Ce système, bien que coûteux en calcul, reste performant grâce à plusieurs optimisations : pré-allocation des nœuds, marquage 4D pour éviter les revisites, et timeout strict à 0.9 seconde.

Le résultat est un pilote raisonnable, qui ira finir toutes ses courses, en ralentissant si il le faut.

5. Points forts et spécificités du pilote

5.1 Un pilote qui fini ses courses

Le vrai point fort de notre pilote est la sécurité. Il est programmé pour finir ses courses. Et si la course est trop longue, ou s'il a percuté un autre pilote et qu'il ne peut plus finir la course, il ira le plus loin possible.

5.2 Une recherche A* optimisée pour le temps réel

Notre A* explore un espace d'état en quatre dimensions (position + vitesse), ce qui pourrait devenir très lourd si on n'y prenait pas garde. Pour éviter ça, on a mis en place plusieurs optimisations :

Pré-allocation d'un pool de nœuds : aucun malloc pendant la course, tout est déjà prêt.

Marquage 4D intelligent : au lieu de réinitialiser un tableau géant à chaque tour, on utilise un identifiant de recherche qui change à chaque itération.

Heuristique Dijkstra inversée : calculée une seule fois au démarrage, elle guide A* de manière fiable sans jamais surestimer la distance.

Ces optimisations permettent au pilote de rester sous la limite d'une seconde par tour, même sur des cartes assez grandes.

De plus, on a bloqué le temps de calcul à 0,9s. Une fois ce temps de calcul dépassé, le pilote va à la case la "mieux" parmi ce qu'il a calculé jusqu'à lors. C'est primordial pour ne pas passer son tour.

5.3 Une architecture modulaire et facile à faire évoluer

Le découpage du code en modules indépendants (communication, carte, physique, pathfinding...) rend le pilote facile à maintenir et à améliorer.

On peut modifier la stratégie, ajouter les boosts, ou intégrer une gestion plus fine des adversaires sans casser le reste du programme. Cette modularité est un vrai atout pour les futures versions du pilote.

6. Difficultés rencontrées et solutions apportées

6.1 Gestion du carburant

Le premier gros problème rencontré a été la gestion du carburant. Notre pilote allait assez vite, mais ne finissait pas les grandes courses. Pour éviter cela, nous associons à chaque état 4D le meilleur niveau de carburant rencontré. Avant d'insérer un nouvel état dans la file de priorité, nous vérifions s'il a déjà été visité avec un carburant supérieur ou égal. Si c'est le cas, l'état est écarté. À l'inverse, si le nouvel état dispose d'un carburant plus élevé, il est conservé et la table est mise à jour.

6.2 Coût du A* en temps

Au début, le A* en 4D coûtait trop cher en temps. L'allocation de tous les noeuds prenait trop de temps et le délais imposé de 1 seconde n'était pas respecté. Donc pour remédier à ce problème on ne fait aucun malloc pendant la course. Nous avons également implémenté une limite de temps interne pour ne pas être surpris par celle du gdc. À chaque itération de la boucle A*, on compare le temps CPU écoulé avec `clock()` de `time.h` à $0,9 \times \text{CLOCKS_PER_SEC}$. Quand ce seuil est dépassé, on sort de la boucle et on renvoie le meilleur chemin trouvé jusque-là.

6.3 Gestion des boosts

La gestion des boosts nous a posé problème. En effet, nous avons tenté, d'implémenter les boosts dans le A*: au lieu de tester l'accélération entre -1 et 1, on a élargi à -2, 2 pour prendre en compte le boost. Néanmoins, en pratique, ce pilote était moins efficace que celui sans boost. Il consommait plus au début, ce qui n'est pas compatible avec notre algorithme: le fait de consommer plus, le fait ralentir sur le reste de la course. Nous avons donc simplement décidé d'abandonner les boosts.

7. Résultats et validation

7.1 Méthodologie de test

Pour valider le comportement de notre pilote, nous avons réalisé une série de tests sur plusieurs cartes fournies par le GDC. L'objectif n'était pas seulement de vérifier que le pilote atteignait l'arrivée, mais aussi d'observer la cohérence de ses trajectoires, sa gestion du carburant et sa capacité à s'adapter aux situations imprévues (collisions, sable, virages serrés...). Cette partie a été très importante sur l'évolution de notre pilote, en réfléchissant directement par rapport à ses courses.

Les tests ont été effectués en conditions réelles. Nous avons lancé le pilote directement dans le GDC. Cela nous a permis d'évaluer son comportement dans un environnement identique à celui de la compétition.

7.2 Stabilité du comportement

Globalement, le pilote présente un comportement stable :

- il ne sort jamais de la piste,
- il gère correctement les collisions en réinitialisant sa vitesse,
- Il respecte les limites de vitesse imposées par le terrain,
- Il finit toutes les courses, en théorie.

7.3 Qualité des trajectoires

Les trajectoires produites par le pilote sont généralement propres et réalistes.

On observe notamment :

- des entrées de virage progressives pour les prendre à la corde,
- des lignes droites bien exploitées,
- une bonne anticipation des zones de sable,
- une vitesse adaptée à la géométrie de la piste.

7.4 Limites

Malgré ces résultats positifs, certaines limites persistent :

- Le pilote n'utilise pas les boosts, ce qui le rend moins performant sur les longues lignes droites. Mais permet
- Il ignore les adversaires dans sa planification,
- On pourrait gagner en vitesse si on se préoccupait moins du gaz, mais c'est une autre stratégie de course,
- Le comportement peut devenir très conservateur lorsque le carburant est bas.
- Ces limites ne remettent pas en cause la stabilité du pilote, mais elles ouvrent des pistes d'amélioration pour les futures versions.

8. Conclusion

Ce projet nous a permis de concevoir un pilote autonome et fiable. Il repose sur une recherche A* dans un espace 4D (x, y, v_x, v_y), couplée à une heuristique Dijkstra inversée pré-calculée. La physique est modélisée fidèlement, la consommation de carburant est intégrée comme contrainte dans l'exploration, et le temps de réponse reste sous 0,9 s sur toutes les cartes testées.

Ce challenge nous a confrontés à des problématiques concrètes : gestion de la mémoire, espace d'état complexe, contraintes temps-réel. Ces exigences ont guidé chacun de nos choix techniques et nous ont poussés à structurer notre code de manière rigoureuse.

Des améliorations restent envisageables. L'intégration des boosts, la prédiction des adversaires sur N tours, ou encore une heuristique 4D intégrant la vitesse permettraient de réduire le

nombre de nœuds explorés. Le pilote actuel forme quand même une base solide, modulaire et évolutive, sur laquelle une version plus ambitieuse pourrait être construite.